

Seeing is Fixing: Cross-Modal Reasoning with Multimodal LLMs for Visual Software Issue Fixing

Kai Huang*, Jian Zhang[†], Xiaofei Xie[‡], Chunyang Chen*

*Technical University of Munich, [†]Nanyang Technological University, [‡]Singapore Management University
{kai-kevin.huang, chun-yang.chen}@tum.de, jian_zhang@ntu.edu.sg, xfxie@smu.edu.sg

Abstract—Large language model (LLM)-based automated program repair (APR) techniques have shown promising results in resolving real-world github issue tasks. Existing APR systems are primarily evaluated in unimodal settings (e.g., SWE-bench), relying solely on textual issue descriptions and source code. However, these autonomous systems struggle to resolve multimodal problem scenarios (e.g., SWE-bench M) due to limitations in interpreting and leveraging visual information. In multimodal scenarios, LLMs need to rely on visual information in the graphical user interface (GUI) to understand bugs and generate fixes. To bridge this gap, we propose GUIRepair, a cross-modal reasoning approach for resolving multimodal issue scenarios by understanding and capturing visual information. Specifically, GUIRepair integrates two key components, Image2Code and Code2Image—to enhance fault comprehension and patch validation. Image2Code extracts relevant project documents based on the issue report, then applies this domain knowledge to generate the reproduced code responsible for the visual symptoms, effectively translating GUI images into executable context for better fault comprehension. Code2Image replays the visual issue scenario using the reproduced code and captures GUI renderings of the patched program to assess whether the fix visually resolves the issue, providing feedback for patch validation. We evaluate GUIRepair on SWE-bench M, and the approach demonstrates significant effectiveness. When utilizing GPT-4o as the base model, GUIRepair solves 157 instances, outperforming the best open-source baseline by 26 instances. Furthermore, when using o4-mini as the base model, GUIRepair can achieve even better results and solve 175 instances, outperforming the top commercial system by 22 instances. This emphasizes the success of our new perspective on incorporating cross-modal reasoning by understanding and capturing visual information to resolve multimodal issues.

Index Terms—Large Language Model, Automated Program Repair, Autonomous Programming, Multimodal Issue

I. INTRODUCTION

Automated program repair (APR) techniques [1]–[6] aim to automatically resolve software defects, reducing manual effort and improving software quality [7]. With the advent of the large language model (LLM) era, LLM-driven APR techniques [5] show promising results in the software engineering (SE) community [8], [9]. Researchers have proposed many autonomous systems to repair software bugs [10]–[14]. Recently, the APR community has begun to focus on a new challenge, SWE-bench [15], designed to evaluate LLMs in solving real-world GitHub issue instances [16]–[21]. These issues are typically represented by textual components including natural language and programming language, few of which contain an image (i.e., screenshot). However, many real-world software engineering

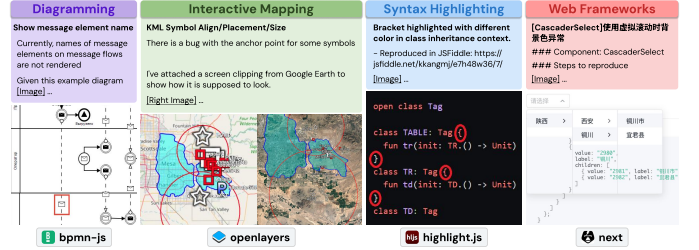


Fig. 1: Four visual task instances from SWE-bench M [22].

tasks involve visual assets, and widely-used front-end libraries enable users to build applications with graphical user interfaces (GUIs). As shown in Figure 1, this is especially common in domains such as web development and interactive application design. When bugs occur in such applications, users often report issues in the repository, accompanied by both textual descriptions and screenshots. Developers maintaining these libraries rely not only on the code and issue descriptions but also heavily on visual cues from the screenshots to understand and diagnose the problems. Without visual context, it becomes difficult to identify the root cause of the issue or to verify whether it has been resolved.

Unfortunately, there has been limited research on automated repair for visual software issues, despite their practical significance. Existing approaches [10]–[14], [16]–[21], [23] primarily target unimodal scenarios that involve only textual information, such as code and natural language. These methods are typically evaluated on benchmarks like SWE-bench [15], Defects4J [24], and HumanEval [25], which are not designed to address multimodal tasks involving both visual and textual elements [22]. Recent findings [22] reveal that top-performing systems on unimodal benchmarks struggle significantly when applied to multimodal settings. For example, SWE-agent [17], which performs strongly on SWE-bench, successfully solves only 12% of tasks in the multimodal SWE-bench M benchmark. This sharp performance decline highlights the unique challenges of multimodal scenarios and underscores the urgent need to advance existing APR systems to better handle visual problems.

In order to explore and reveal the reasons behind this, we analyze the workflows of existing AI systems and identify two key limitations that hinder their effectiveness in visual issues: **1) Insufficient Understanding of Visual Issues.** Most LLM-driven autonomous systems focus on analyzing textual issue descriptions and source code to identify and repair defects. However, they often overlook the relationship between visual

[†]Corresponding author: Jian Zhang (jian_zhang@ntu.edu.sg).

outputs (e.g., screenshots) and the underlying code components. Directly incorporating images from issue reports poses challenges—LLMs typically lack the domain-specific context needed to interpret visual scenarios. They may struggle to identify which code components are responsible for the visual issue or how these components interact to produce the faulty behavior. Consequently, this limited understanding of visual symptoms can hinder accurate fault localization and reduce the effectiveness of generated patches. **2) Lack of Feedback for Repair Behaviors.** Existing autonomous systems typically rely on executing test suites to validate and select valid patches [10]–[12]. However, SWE-bench M omits test cases to prevent potential data leakage, removing a critical feedback signal for patch validation. While some approaches [17], [18] generate test cases as substitutes, they are not well-suited for visual issues. On the one hand, certain visual problems rely on pixel-level comparison in visualization-based test cases [22], which are challenging for language models to generate accurately. On the other hand, although reproducing issue scenarios can provide feedback on patch effectiveness [17], [26], most multimodal task instances in SWE-bench M lack reproduction scripts [22], making it difficult to recreate the necessary visual conditions. As a result, a key challenge is how to reliably reproduce visual scenarios to enable effective patch validation.

In this paper, we propose GUIRepair, a cross-modal transformation solution designed for multimodal scenarios. GUIRepair addresses the challenges of multimodal issue resolution by enabling seamless transformation between textual and visual information, thereby facilitating both the understanding and utilization of visual content. Specifically, GUIRepair comprises two core components: Image2Code and Code2Image. For simplicity, the term *image* is used to refer to a screenshot in this paper. Image2Code enhances bug comprehension by interpreting visual information to support accurate fault localization, while Code2Image generates visual representations from code to provide feedback for patch validation. **1) Image2Code:** Before performing fault localization, GUIRepair first selectively retrieves relevant project documentation based on the multimodal issue report to better understand the visual problem. It then uses these retrieved documents as domain knowledge within the knowledge injection paradigm to generate the code that reproduces the visual issue. This process transforms high-level visual information into its corresponding code behavior, providing essential contextual knowledge to map the visual problem to specific code components. In essence, by reasoning from visual issue scenarios to their textual code representations, GUIRepair enables large language models to develop a more comprehensive understanding of the problem using both visual and textual features. **2) Code2Image:** After generating patches, GUIRepair replays the bug using the previously generated reproduction code and captures the resulting behavior in the GUI. It then evaluates whether the visual presentation of the patched program aligns with the expected outcome described in the issue report, thereby filtering for potentially correct patches. This step provides critical feedback for patch validation by converting the abstract fixing behavior into concrete,

observable visual effects. That is, by translating textual repair behaviors into visual outcomes, GUIRepair enables language models to verify patch effectiveness more intuitively through multimodal cues. Finally, these two core components are seamlessly integrated into the overall repair workflow including fault localization, patch generation and patch validation. This bidirectional transformation enables GUIRepair to reason across visual and programmatic modalities, significantly enhancing the accuracy and effectiveness of automated issue repair.

We evaluate GUIRepair on the SWE-bench M benchmark comparing against state-of-the-art autonomous systems. Our results show that GUIRepair is able to achieve the best performance (157 resolved instances) than other open-source approaches when keeping the same base model GPT-4o. In particular, GUIRepair’s two key components with Image2Code and Code2Image resolved 21 more instances than the basic repair workflow, marking a 15.44% improvement. Furthermore, when using the advanced reasoning model o4-mini, GUIRepair can achieve amazing results (175 resolved instances), which solves 22 instances more than the best commercial system.

Overall, the main contributions of this paper are as follows:

- **Dimension.** To the best of our knowledge, this is the first work to explicitly address issues with visual cues in front-end libraries, which opens a new dimension in automated issue resolution by incorporating textual and visual modalities.
- **Technique.** We propose GUIRepair, a bidirectional cross-modal transformation approach to solve multimodal software development problems. GUIRepair performs image-to-code transformation to diagnose issues by extracting semantic features from GUI screenshots and mapping them to code-level representations. Furthermore, it applies code-to-image transformation to validate repair patches by rendering the updated GUI and assessing its visual correctness.
- **Evaluation.** We conduct extensive experiments on the popular SWE-bench M benchmark, comparing GUIRepair against a range of open-source and commercial autonomous systems. GUIRepair resolves 157 issues, outperforming all existing approaches. Notably, its two key components (Image2Code and Code2Image) contribute a 15.44% improvement in overall effectiveness, highlighting the impact of cross-modal reasoning in addressing complex multimodal program repair tasks. We have released our code and experimental data [27].

II. MOTIVATION

In this section, we analyze the gap of existing autonomous systems in resolving visual software problems, which motivates us to design a cross-modal reasoning approach.

A. Challenge 1: Fault Understanding.

In unimodal scenarios, user-described issue symptoms are typically conveyed through text [15]. However, multimodal issue reports often involve both textual and visual elements [22]. As shown in Figure 2, these reports usually contain three layers of information: 1) natural language descriptions, 2) visual issue images, and 3) reproduced code that links symptoms to underlying components. Text alone often fails to capture visual



Fig. 2: A motivating example of alibaba-fusion/next-895 [29].

issues. A recent study [22] found that 80% of visual symptoms cannot be fully described in text, and images are essential for issue-solving in 83.5% of cases.

Even when text and images are available, models may still struggle to analyze visual symptoms of users' applications and identify root causes in the used front-end library. This is because LLMs may lack human expert knowledge about the specific front-end project [28], making it difficult for them to understand the code behavior behind the visual information. This gap stems from the lack of reproduced code, which clarifies the relationship between visual symptoms and code behavior. Yet, only 17% of SWE-bench M reports include such code [22], leaving most cases without sufficient context. This highlights a key challenge in multimodal bug resolution: inferring code behavior from visual symptoms for effective fault localization.

B. Challenge 2: Patch Validation.

AI systems typically generate multiple candidate patches, but only the top-ranked one is selected in order to best meet developers' needs (e.g., SWE-bench uses the Pass@1 metric). Therefore, accurately identifying which patches are effective is a crucial post-processing step. However, most current systems typically cannot perform patch validation on visual scenes directly [30], limiting their ability to provide meaningful feedback on patch correctness.

While some approaches validate patches in unimodal settings using textual test cases [18], generating reliable visual oracles that involves pixel-level accuracy remains difficult, especially for issues requiring visual testing [22], [31], [32]. For example, data visualization and interactive mapping frameworks like *Chart.js* [33] and *OpenLayers* [34] often rely on pixel-level visual tests to verify correctness. As shown in Figure 3, we illustrate a *Chart.js* issue [35]: *borderRadius* is ignored for the bottom corners of 0-value bars when *borderSkipped* and *minBarLength* are set. Different proposed fixes (PR 1–N) produce subtle but distinct visual effects. Therefore, unlike in textual domains, generating such image test cases is infeasible for LLMs, as they cannot produce accurate visual references. This highlights a central challenge in visual patch validation: capturing and assessing visual effects as the repair feedback.

III. APPROACH

In this section, we introduce GUIRepair to solve visual software issues by reasoning the correlation between visual information from user-submitted reports and code components

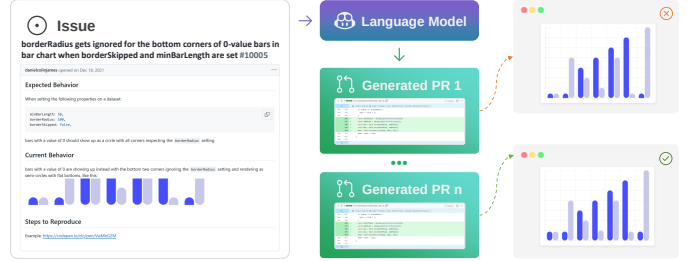


Fig. 3: A motivating example of chartjs/Chart.js-10157 [35].

from the front-end library. Basically, GUIRepair follows the traditional APR pipeline [3], which is organized into four main steps: Fault Comprehension, Fault Localization, Patch Generation, and Patch Validation. Figure 4 illustrates the overall workflow. In the fault comprehension phase (①–②), GUIRepair first mines project-specific knowledge and generates reproduction code to help LLMs better understand the issue of the library. This reproducible code is essential for replicating the problem scenario in user applications. In the fault localization phase (③–④), GUIRepair leverages the knowledge injection paradigm [36], utilizing the domain knowledge acquired in the previous phase to assist LLMs in accurately identifying the code components associated with specific visual elements. During the patch generation phase (⑤), GUIRepair prompts the LLM to perform multiple sampling rounds, producing candidate patches for the library based on the previously learned knowledge of the bug scenario. Finally, in the patch validation phase (⑥–⑦), instead of generating test cases, GUIRepair captures the visual effects of generated patches by rendering GUI of the reproduced code and provides feedback to select the most promising patch.

A. Fault Comprehension

As highlighted in Section II-A, existing AI systems often struggle with multimodal software engineering (SWE) tasks due to limited understanding of the interplay between visual and code representations. This lack of multimodal comprehension impairs effective fault localization. To address this challenge, GUIRepair introduces the Image2Code component, which enhances fault comprehension by enabling models to acquire project-specific domain knowledge and reason about the code components behind visual scenarios. As shown in Figure 4, Image2Code comprises two key steps: ① Knowledge Mining, and ② Reproduction Generation.

1) *Knowledge Mining*: To facilitate effective reasoning, it is essential for the LLM to familiarize itself with the relevant project context such as usage patterns [37], so as to act more like a human developer. As shown in Figure 5a, project documents (docs) contain important domain knowledge about what components are in the project and how to use them. Hence, GUIRepair collects project-specific knowledge and incorporates it using the knowledge injection paradigm [36], enabling the model to better comprehend the problem in subsequent reasoning tasks. As illustrated in Figure 4, GUIRepair takes both the issue report and project documentation as input. By processing

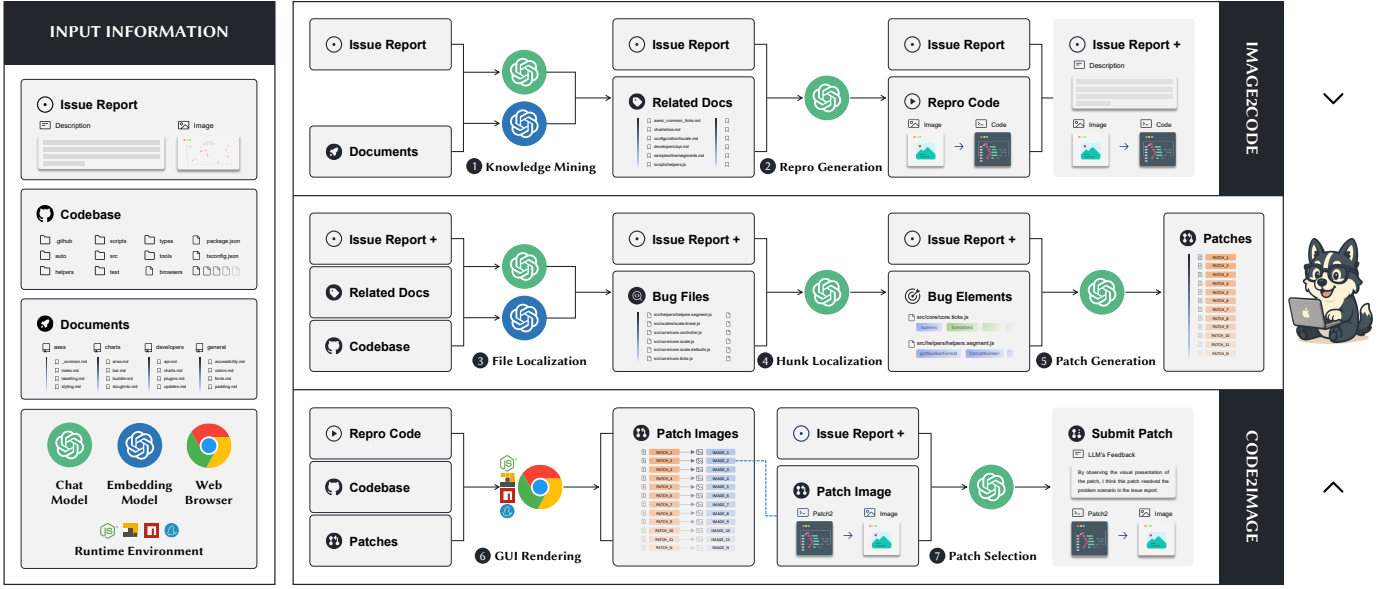
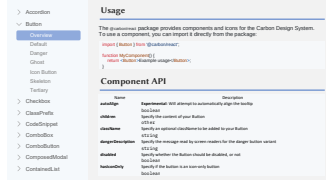


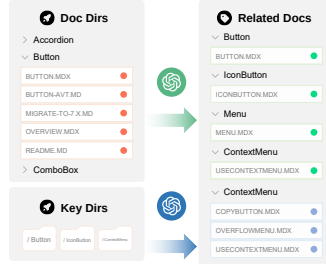
Fig. 4: The overview of GUIRepair. Note: The left shows the system input, and the right illustrates the step-by-step repair workflow.

the issue report, the model heuristically identifies and selects documents that are most relevant to the described scenario.

Specifically, we first prompt the chat model (e.g., GPT-4o [38]) to analyze the issue report and, based on its initial understanding, identify potentially relevant documents from the project’s documentation directory. Due to the limited context window of LLMs, it is impractical to load and process all documents at once. Inspired by how human developers browse documentation, we instead provide only the directory structure, allowing the model to select the Top-N most relevant document



(a) Example of doc content.



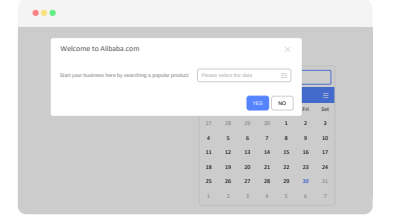
(b) Example of mined docs.

Fig. 5: Example of documents. filenames. However, selecting documents solely based on filenames may miss important content, as the model cannot access the full text during this step. To address this, and inspired by retrieval-augmented generation (RAG) techniques [39], we further use an embedding model (e.g., text-embedding-3 [40]) to retrieve relevant content from the documentation repository. This retrieval is guided by the issue report and the chat model’s preliminary understanding. To reduce retrieval costs in large projects with hundreds or thousands of files, we limit the search to key directories identified in the previous chat model phase and return the Top-N documents with the highest similarity scores. Ultimately, we obtain two sets of relevant documents: one selected by the chat model through directory browsing, and another retrieved via the embedding-based similarity search. In Figure 5b, these two sets are merged to form a final collection

of contextually relevant documents, which helps LLMs acquire project knowledge related to the issue scenario.

2) *Repro Generation*: Inspired by website generation [41], [42] and bug reproduction [26], we design the reproduction (repro) generation step to enhance the model’s understanding of visual problems by generating the corresponding code that reproduces the issue. To support this, we reorganize the model input by incorporating previously mined relevant documents as domain knowledge. This domain knowledge, which is closely related to the issue scenarios, is injected directly into the prompt as reference material [36].

As shown in Figure 6, we present the repro code behind a visual issue *next-1509* [43]. Figure 6a shows the symptoms of the visual issue, where the calendar UI is incorrectly displayed below the dialog box, preventing users from directly using the calendar. The repro code in Figure 6b reveals which code components were called in the visual problem (line 1-2) and how these code components were organized to trigger the problem scenario (line 4-47). These code details in Figure 6b are crucial for understanding the visual problem and pinpointing the relevant code elements. The goal of this step is to enable the model to learn project-specific knowledge in a manner similar to a human



(a) The visual issue scenario of *next-1509*.



(b) The repro code in front-end project to replay the issue scenario of *next-1509*.

Fig. 6: Example of the reproduced code behind the front-end visual issue scenario (Image2Code).

The goal of this step is to enable the model to learn project-specific knowledge in a manner similar to a human

expert, thereby allowing it to reconstruct the code behind the visual issue. In many multimodal scenarios, reproducing an issue requires building a complete front-end project, typically composed of HTML, CSS, and JavaScript (JS) files [22]. Among these, the JS file contains the dynamic logic that reproduces the visual problem, while the HTML and CSS files generally contain static content that does not require frequent changes. Therefore, we instruct the model to generate only the JS code needed to reproduce the multimodal scenario. This output reveals the code components responsible for the visual problem and illustrates how they are structured to trigger the faulty behavior. The generated code (as shown in Figure 6b) is then embedded into the original issue report. The resulting enhanced issue report (i.e., Issue Report+), now includes the natural language description, the programming language reproduced code, and the visual representations of the issue. The enhanced issue report can be used in subsequent localization, generation, and validation processes to enhance fault understanding and implement fixes. Indeed, an LLM may not always generate completely accurate reproduction code. Minor discrepancies may exist between the generated output and the original visual report, such as differences in button sizes or text highlight colors. However, this is tolerable, as our primary goal is to expose the underlying code behavior responsible for the visual problem. As long as the reproduced code sufficiently captures the essential behavior of the issue (e.g., API misuse), it serves its purpose in helping the model understand the multimodal scenario. Furthermore, during the patch validation phase, we will re-render the issue scenario in a controlled local environment using the generated reproduction code. This eliminates noise from non-essential elements, allowing the LLM to focus more effectively on the core fault. As a result, minor detail discrepancies in the reproduced code have minimal impact on the repair process.

B. Fault Localization

After obtaining the reproduced code and related documents for the visual problem, these project-specific domain knowledge will be used to enhance the fault localization process. As shown in Figure 4, fault localization in GUIRepair is a two-stage process: ③ file localization and ④ hunk localization.

1) *File Localization*: In the file localization phase, GUIRepair leverages enhanced issue reports (with reproduced code) to understand the fault scenario and identify potentially related code files within the codebase. To achieve this, we inject project-specific documents into the chat model, enabling it to analyze the enhanced issue report and identify files likely to contain the bug. Since having the LLM read the entire codebase would exceed its context limit, we follow Agentless to extract the *repository structure format* [18]. Based on this structure, the model is prompted to return the paths of suspicious files after reviewing the overall layout of the codebase. In addition, we use the embedding model to retrieve Top-N relevant code files from the codebase based on the enhanced issue report, helping to mitigate the chat model’s limitation of not being able to view file content directly during the file localization phase. Similar to

the document retrieval strategy described in Section III-A1, the embedding model restricts its search to the directories identified by the chat model as likely containing the bug, which reduces the overhead of scanning the entire codebase by focusing only on files within key directories. Finally, we merge the suspicious file paths identified by both the chat model and the embedding model to form the final set of suspicious files.

Note that the final set may include a dozen or more files, which can exceed the model’s context limit during the hunk localization. To address this, we set a maximum number N of key bug files. If the file set exceeds this limit, we filter and retain only the Top- N most relevant files for further analysis. In detail, we provide the LLM with the code content of each suspicious file and ask it to select Top- N key bug files. Inspired by the *skeleton format* [18], we input a streamlined format of each file to avoid context limits. The original skeleton format retains class/function headers, variable declarations, and comments. GUIRepair extends this by preserving import statements to capture file-level dependencies and removing variable declarations to streamline the input. Based on this lightweight representation and its understanding of issue scenarios, the chat model identifies the Top- N key bug files.

2) *Hunk Localization*: In the hunk localization phase, we start with the list of key bug files and aim to identify the specific bug code hunks. We localize at the class or function level, rather than pinpointing exact code edit locations [18]. This higher-level granularity simplifies the process while still capturing meaningful fault contexts. Specifically, we provide the chat model with the full contents of all key bug files, allowing it to identify specific classes or functions that are likely responsible for the issue based on its understanding of the enhanced issue report. Unlike earlier stages where a compressed skeleton format is used, here we retain the complete file content. This is necessary because some bugs may appear outside of classes or functions, such as in configuration files or global code blocks, and the model needs full context to accurately navigate and localize the fault. To alleviate context limitations, we compress only the content within classes and functions by defining a maximum hunk size of N lines. If a class or function exceeds this limit, we retain only its header, along with internal variable declarations and comments. The model then returns the names of all suspected bug-related classes or functions. For each identified element, we extract its complete class or function body as the localized hunk. If the bug is located outside of a class or function such as in a configuration block or global scope, the model returns the relevant element name. In these cases, we define a context window (e.g., N lines) and extract a surrounding code segment that includes the bug element, which is then used as the localized hunk.

C. Patch Generation

In the ⑤ patch generation process, our goal is to generate candidate patches for the bug code snippets localized in the previous phase. Here, we follow the learning-based APR work [12]–[14], [25], [44]–[65] to input class/function level bug code snippets and then ask the chat model to generate patches



(a) The bug/patch_1/patch_2 code behaviors in *prism-1602*.



(b) The visual images effects of bug/patch_1/patch_2 code in *prism-1602*.

Fig. 7: Example of front-end visual image effects corresponding to bug/fix code behaviors (Code2Image).

directly. For the format of the generated patches, we adopt the *Search/Replace edit* strategy used in Agentless [18], due to its simplicity and effectiveness. A *Search/Replace edit* consists of two key components: (1) a *search* string, representing the original buggy code to be replaced, and (2) a *replace* string, indicating the code snippet that should take its place. To integrate the patches into the project codebase, we search for occurrences of the specified *search* snippet within the source files and replace them with the corresponding *replace* snippet. This step enables seamless backfilling of the generated patches into the codebase. Finally, we convert each patch into the unified git diff format using the *git diff* command. These diff-formatted patches provide a clearer picture of the change behaviors and are used in the subsequent patch validation.

D. Patch Validation

In the LLM-driven APR workflow, LLMs typically sampled multiple times [62]–[65] to generate a diverse set of candidate patches. However, in the SWE task, a robust system is expected to achieve high accuracy using only its top-ranked patch, as measured by the Pass@1 evaluation metric. This requirement more accurately reflects the problem-solving capability of the AI system. Therefore, the goal of the patch validation phase is to identify and select the most promising patches from the generated patch space. To facilitate this process, we introduce the Code2Image component, which assesses patches by capturing the visual representation of repair behavior. The validation process consists of two phases: ⑥ GUI rendering, and ⑦ patch selection.

1) *GUI Rendering*: In this phase, we aim to provide visual feedback for patch validation by capturing how the patch alters the third-party application’s behavior as rendered in the GUI. To achieve this, we leverage the reproduced code generated by the LLM during the fault comprehension phase to construct a runnable front-end project. This project is then executed in a web browser to visually reflect the effect of the applied patch within the original bug scenario. As shown in Figure 7, different repair behaviors can produce different visual effects on the front end. Although automatically replicating the execution environment would be ideal, it involves installing complex

dependencies and adjusting JS package imports, which is an error-prone task for LLMs. As a practical compromise, we follow the procedure used in the SWE-bench M study [22] and perform this process manually. First, we manually set up the local runtime environment for each JS project by following the repository’s developer or contributor guidelines and installing the necessary package manager and dependencies. Second, we build front-end projects to replay the bugs using the reproduced code, manually importing the built JS package into the HTML file. We then run the reproduced code in a local browser to capture a screenshot of the bug scenario. In some cases, minor manual modifications are needed to adjust package paths for correct execution. Third, we apply each candidate patch from the patch space to the buggy repository. For each patched version, we rebuild the project to obtain a new JS package and run it in the browser. The resulting visual output is captured to represent the behavior of each patch. Finally, we obtain one bug scenario image and multiple patch scenario images.

2) *Patch Selection*: In the patch selection phase, we evaluate the effectiveness of each patch using the previously captured visual information. Specifically, we first perform pixel-level comparisons between each patch scenario image and the bug scenario image. Patches that result in no visual change are considered ineffective and are filtered out. The remaining patch scenario images are then sequentially input to the LLM, which assesses whether the visual output reflects a successful fix based on its understanding of the issue report. Once the model identifies a patch as effective, the validation process terminates. The validated patch is then retained as the resolution for the issue. To illustrate this process, consider the *prism-1602* issue [66], where YAML strings fail to highlight when a comment appears on the same line (i.e., “world”). The buggy code (Figure 7a) results in incorrect rendering in the front-end application (Figure 7b). Our tool initially generates two candidate fixes: Patch_1 and Patch_2. Patch_1 disables syntax highlighting entirely, while Patch_2 correctly restores it. By observing these visual differences and associating with the previous understanding of issue symptoms, the LLM selects Patch_2 as the effective fix. This example demonstrates how visual effects guide the model in choosing appropriate patches.

IV. EXPERIMENT SETUP

A. Research Question

- **RQ1: How well does the overall effectiveness of GUIRepair compare to state-of-the-art SWE autonomous systems?** This question evaluates GUIRepair’s overall capabilities in solving multimodal issues. (Overall Effectiveness)
- **RQ2: How much does the design choice of GUIRepair contribute to the overall repair capability?** This question explores how Image2Code and Code2Image components affect the overall repair performance. (Ablation Study)
- **RQ3: Can GUIRepair generalize to additional multimodal task instances as well as other base models?** This question assesses GUIRepair’s generalization ability on additional repositories and other models. (Generalizability Study)

B. Benchmark

We evaluate GUIRepair in SWE-bench M [22], which is a benchmark for evaluating AI systems to fix bugs in visual, user-facing JavaScript software. It contains 619 task instances from 17 popular JavaScript libraries used for web interface design, diagramming, data visualization, syntax highlighting, and interactive mapping, each featuring images crucial to problem-solving. SWE-bench M is divided into two parts: Test split (SWE-bench M test) contains 517 instances. Development split (SWE-bench M dev) contains 102 instances. We use the standard test benchmark SWE-bench M test to evaluate the overall effectiveness, and the additional development set SWE-bench M dev for conducting the generalizability study.

C. Baseline

We compare GUIRepair against recent 13 SWE systems. These baseline tools represent the state-of-the-art performance on SWE-bench M. We directly select the top representative techniques on the official SWE-bench M leaderboard [67] as of May 2025 (around the paper submission time) for comparison. We include open-source as well as commercial or closed-source baselines, and the recent Computer Use Agent [68]. We additionally provide a GPT-4o [38] based implementation for Agentless Lite [30] to serve as a complementary baseline, it is currently the best open-source system on SWE-bench M.

D. Implementation

We follow recent SWE studies [18], [22] to chose gpt-4o-2024-08-06 [38] as the chat model and text-embedding-3-small [40] as the embedding model for the implementation of GUIRepair. This is because GPT-4o solves more task instances at lower repair costs [22] and facilitates a fair comparison with baselines using a unified base model. In terms of parameter settings, we follow the Agentless experience [18], [30]. By default, we query the chat model with greedy decoding (i.e., temperature = 0). In the ❶ knowledge mining phase, we set the temperature for the chat model to 0, the sampling times to 1, and let the model return the Top-6 relevant documents. Then, we set the chunk size for the embedding model to 512, chunk overlap to 0, and select the Top-6 relevant documents retrieved by the model. Finally, we merge the results of the above two parts to get the final document list. In the ❷ repro generation phase, we use the default chat model settings and one-shot prompting to generate the reproduced code. In the ❸ file localization phase, we set the chat model’s temperature of 1 and sampling time of 2 to obtain a diverse distribution, and ask the model to return all suspicious files. Besides, we keep the same settings for embedding model just like in the knowledge mining phase, and return Top-4 bug files. Similarity, we merge the results of the above two parts to get the final file list. In particular, we set the maximum number of candidate bug files to 4. If the final file list exceeds the maximum number of files, we ask the chat model (default settings) to read these candidates and filter out the Top-4 key files to alleviate the context limit. In the ❹ hunk localization phase, we set the temperature of 1 and sampling time of 2 to cover potential bug code snippets/hunks

TABLE I: Results on SWE-bench M test. Note: 🗑️ refers closed-source commercial systems. 🍌 refers claude-3-5-sonnet-20240620. 🟢 refers gpt-4o-2024-08-06.

SWE System	Base Model	Resolved	%Resolved	\$Avg. Cost
🗑️ Globant Code Fixer Agent [70]	NA	153	29.59%	-
🗑️ Zencoder [71]	NA	140	27.08%	-
🍌 Agentless Lite [30]	🍌 Claude3.5	131	25.34%	\$0.25
Agentless Lite [30]	🟢 GPT-4o	127	24.56%	\$0.18
Computer-Use Agents [68]	🟢 GPT-4o	104	20.12%	-
SWE-agent Multimodal [22]	🟢 GPT-4o	63	12.19%	\$2.94
SWE-agent [17]	🍌 Claude3.5	63	12.19%	\$1.52
SWE-agent JavaScript [22]	🍌 Claude3.5	62	11.99%	\$3.11
SWE-agent [17]	🟢 GPT-4o	62	11.99%	\$2.07
SWE-agent Multimodal [22]	🍌 Claude3.5	59	11.41%	\$3.11
SWE-agent JavaScript [22]	🟢 GPT-4o	48	9.28%	\$0.99
Agentless [18]	🍌 Claude3.5	32	6.19%	\$0.42
RAG [22]	🟢 GPT-4o	31	6.00%	\$0.17
RAG [22]	🍌 Claude3.5	26	5.03%	\$0.15
Agentless [18]	🟢 GPT-4o	16	3.09%	\$0.38
🍌 GUIRepair	🟢 GPT-4o	157	30.37%	\$0.29

TABLE II: Resolved multimodal issues by repository.

Repo	Num	GUIRepair 🟢	Globant [70] NA	Zencoder [71] NA	AgentlessL [30] 🍌	AgentlessL [30] 🟢	SWE-agent [22] 🟢
next	39	9 (23.08%)	10 (25.64%)	6 (15.38%)	6 (15.38%)	3 (7.69%)	0 (0.00%)
bpmn-js	54	35 (64.81%)	34 (62.96%)	28 (51.85%)	27 (50.00%)	28 (51.85%)	15 (27.78%)
carbon	134	12 (8.96%)	12 (8.96%)	15 (11.19%)	9 (6.72%)	9 (6.72%)	2 (1.49%)
eslint	11	6 (54.55%)	2 (18.18%)	2 (18.18%)	2 (18.18%)	1 (9.09%)	0 (0.00%)
lighthouse	54	3 (5.56%)	7 (12.96%)	7 (12.96%)	4 (7.41%)	3 (5.56%)	3 (5.56%)
grommet	21	1 (4.76%)	1 (4.76%)	4 (19.05%)	2 (9.52%)	1 (4.76%)	0 (0.00%)
highlight.js	39	4 (10.26%)	4 (10.26%)	4 (10.26%)	3 (7.69%)	1 (2.56%)	1 (2.56%)
openlayers	79	76 (96.20%)	76 (96.20%)	64 (81.01%)	73 (92.41%)	75 (94.94%)	41 (51.90%)
prettier	13	2 (15.38%)	2 (15.38%)	1 (7.69%)	0 (0.00%)	1 (7.69%)	1 (7.69%)
prism	38	7 (18.42%)	4 (10.53%)	7 (18.42%)	5 (13.16%)	5 (13.16%)	0 (0.00%)
quarto-cli	24	2 (8.33%)	1 (4.17%)	2 (8.33%)	0 (0.00%)	0 (0.00%)	0 (0.00%)
scratch-gui	11	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)
Resolved		157	153	140	131	127	63
%Resolved		30.78%	30.00%	27.45%	25.69%	24.90%	12.35%

to provide adequate contexts (or repair ingredients) for patch generation. In particular, for bug elements that are not inside the class/function, we set the context window to 500 to intercept the code hunk where the bug element is located. In the ❶ patch generation phase, we use the greedy decoding strategy (default settings) to generate a patch and use the multi sampling strategy (temperature is 1 and sampling times is 39) to return 39 patches. Finally, we will get at most 40 patch candidates. In the ❷ image capturing phase, we use *fnm* to control different *Node.JS* versions and use the *npm/pnpm/yarn* as the JavaScript package manager, then we use Playwright [69] to replay/capture the issue/patch scenario in browsers. In the ❸ patch selection phase, we keep the default chat model settings and only submit the Top-1 valid patch to the SWE-bench evaluation platform.

V. EVALUATION

A. RQ1: Overall Effectiveness

In the main experiment, we will investigate the overall effectiveness of GUIRepair on the SWE-bench M test. Table I presents the evaluation result of existing AI systems. The effectiveness of GUIRepair in solving multimodal instances exceeds that of all SWE systems currently on the SWE-bench M leaderboard. Specifically, GUIRepair solves 4 more task instances than the best closed-source commercial system, Globant Code Fixer Agent [70], and 17 more task instances than Zencoder [71]. This shows that GUIRepair’s simple workflow is comparable to complex agent systems in terms of repair capability. Besides, GUIRepair also solves 26 more task instances than the best open-source system Agentless Lite [30]. Note that since closed-source systems do not release the base model they use, it is unfair to make a comparison directly with systems implemented with different models. Thus, if keeping

the consistent base model, the GPT-4o based implementation of GUIRepair solves 30 more task instances than the best baseline Agentless Lite. Recall that GUIRepair adds design choices for multimodal scenarios to the agentless workflow. And the fact that our approach achieves better results than the general agentless system [18], [30], which potentially suggests that our novel design further improves the ability of existing agentless systems to tackle visual problems. Meanwhile, compared to agent systems [17], [70], [71], the agentless-based GUIRepair inherits the cost advantage of agentless approaches [18], [30], which requires only \$0.29 on average to solve a multimodal instance. Overall, GUIRepair is a success in terms of overall effectiveness. Not only does it remain relatively inexpensive in repair cost, but it rivals and surpasses state-of-the-art closed-source and open-source systems in terms of repair capability.

Further, we will also explore the difference in repair capability between GUIRepair and existing systems in solving multimodal problems. Here, we present the unique issues resolved by GUIRepair compared to existing Top-5 SWE systems in Figure 8, and analyze the differences in the ability of these systems to resolve different repositories in Table II. As shown in Figure 8, different SWE systems maintain unique fixes, suggesting that the different approaches differ in the characterization of the resolved issues. In particular, although GUIRepair is very close to the state-of-the-art closed-source commercial system Globant [70] (Globant Code Fixer Agent) in terms of repair capability (157 vs. 153), they both possess unique issue resolved (12 vs. 12), which suggests that GUIRepair solves task instances that are difficult to solve by the closed-source system, while the closed-source system at the same time compensates for GUIRepair’s shortcomings in terms of repair capability. Moreover, the overall distribution shows that GUIRepair complements rather than duplicates the capabilities of the other systems. In addition, as shown in Table II, different SWE systems have their strengths in solving the issues of different repositories. Specifically, we use the green background to highlight the best results from each repository. GUIRepair, Globant, and Zencoder achieved the best results in 7, 5, and 6 repositories. Overall, GUIRepair is orthogonal to existing systems in terms of repair capability, and it is competitive and complementary compared to top commercial systems.

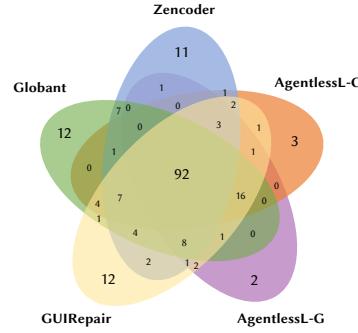


Fig. 8: Venn diagram for multimodal issue fixes. Note: AgentlessL-C/G refers Claude3.5/GPT4o-based Agentless Lite.

B. RQ2: Ablation Study

In the ablation experiment, we aim to evaluate the contributions of GUIRepair’s key components (Image2Code and Code2Image) in enhancing the repair capability. Here, we designed four variants to reveal the effects of design choices:

TABLE III: Results of GUIRepair variants on SWE-bench M.

Variants	Components		Result	
	Image2Code	Code2Image	Resolved	Avg. \$Cost
GUIRepair _{base}	-	-	136	\$0.08
GUIRepair _{I2C}	✓	-	146 (+10)	\$0.10
GUIRepair _{C2I}	-	✓	148 (+12)	\$0.27
GUIRepair _{full}	✓	✓	157 (+21)	\$0.29

- **GUIRepair_{base}**: To provide a basic baseline to present the repair capabilities of the basic agentless approach in multimodal scenarios, we remove two key components (Section III-A and III-D) of the GUIRepair approach and keep only the core fault localization and patch generation modules (Section III-B and III-C). This variant, called GUIRepair_{base}, receives multimodal issue reports and codebases directly as input and generates only one valid patch to be submitted.
- **GUIRepair_{I2C}**: To present the improvement of the Image2Code component for the basic agentless workflow to solve multimodal scenarios, we add the Image2Code to the basic workflow. This variant, called GUIRepair_{I2C}, not only receives visual issue reports and codebases, but also analyzes the code behavior behind the visual information by mining project-specific knowledge to understand visual scenarios.
- **GUIRepair_{C2I}**: To indicate the enhancement of the Code2Image component for the basic agentless workflow to solve multimodal scenarios, we add the Code2Image to the basic workflow. This variant, called GUIRepair_{C2I}, also receives issue reports and codebases as input, in particular, it helps to select potentially correct patches from the patch space by capturing the visual effects of the repair behavior.
- **GUIRepair_{full}**: To explore the contribution of the two key components to overall effectiveness, we refer to the complete method implementation as GUIRepair_{full}. In fact, the two key components of GUIRepair, along with the basic agentless approach, form the complete workflow, and the process is an interdependent one, so that only the simultaneous use of the two key components can maximize the repair benefits.

As shown in Table III, we demonstrate the performance of different variants on SWE-bench M test. Next, we will analyze the impact of design choices on the repair capability.

1) **Basic Capability of the Agentless Workflow**: As shown in Table III, GUIRepair_{base} provides a reference baseline to indicate the performance achieved by the basic agentless workflow. Specifically, GUIRepair_{base} solves 136 instances, which is close to the performance of Agentless Lite (Table I). This is because both GUIRepair_{base} and Agentless Lite follow the basic agentless workflow, and thus they are comparable in terms of repair capability. In particular, we note that GUIRepair_{base} maintains a lower cost compared to existing agentless systems, Agentless Lite [30] and Agentless [18], [22]. This is due to the fact that our implementation further simplifies the fault localization process. Compared to Agentless Lite, which locates faults by retrieving the codebase, GUIRepair_{base} only retrieves the key file directories, thus reducing the file

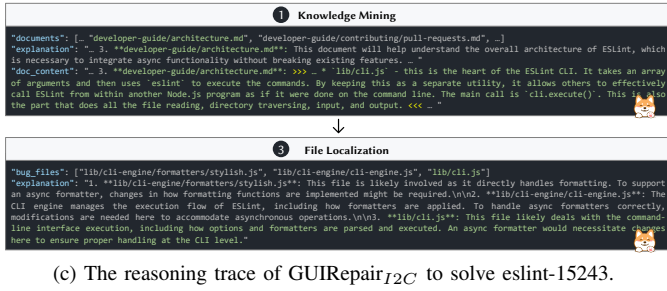
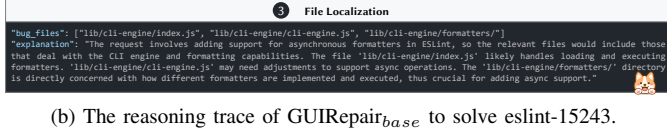
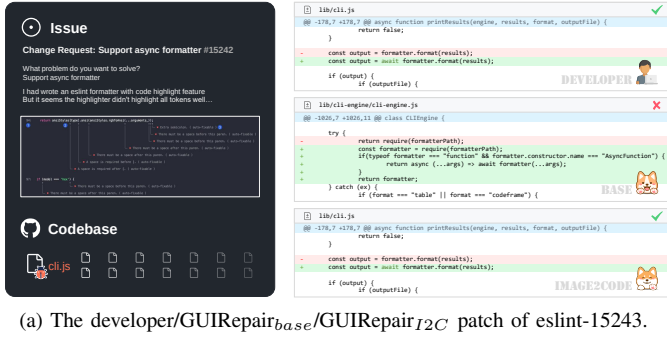


Fig. 9: The case study of eslint-15243.

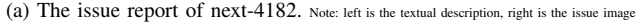
retrieval cost. Compared to Agentless, which uses three levels (file/element/edit-level) of fault location granularity to locate specific edit locations, GUIRepair_{base} uses only two levels (file-level and hunk-level) of fault localization to locate hunk-level code snippets, thus further simplifying the complexity of the localization process. Benefiting from our simplified design, our implementation of the basic agentless approach has a lower cost advantage while maintaining good repair effectiveness.

2) **Improved Capability of the Image2Code Component:** As shown in Table III, GUIRepair_{I2C} presents improvements of the Image2Code component to the basic repair workflow. Specifically, GUIRepair_{I2C} solves 10 more instances than GUIRepair_{base}, achieving a 7.35% improvement. Let's recall that one of the purposes of the Image2Code component is to enhance fault localization by mining the project-specific knowledge to understand the visual information. To further clarify the contribution of the Image2Code, here we provide a case study to analyze how the Image2Code component works. As shown in Figure 9, we demonstrate a instance *eslint-15243*, which was successfully resolved by GUIRepair_{I2C}, but the GUIRepair_{base} fix failed. In Figure 9a, GUIRepair_{I2C} accurately located the bug file *cli.js* and generated a patch consistent with the developer patch. Yet GUIRepair_{base} did not locate the correct bug file, which caused the fix to fail. Further, we try to analyze the reasoning trace behind these two variants to reveal why GUIRepair_{I2C} was able to locate the bug file. As shown in Figure 9b, GUIRepair_{base} reads the issue report directly to locate suspicious files, however, since the model does not have project-specific knowledge, it struggles to

accurately locate the relevant code files behind the visual issue. In contrast, in Figure 9c, GUIRepair_{I2C} first learns the project details in the knowledge mining phase, where it reads the key documents and learns the role of the *cli.js*. Then, benefiting from the fact that the model has learned that project knowledge, when implementing fault localization, the model considers *cli.js* as a suspicious file. In this way, GUIRepair_{base} misses the opportunity to localize to the bug file due to its lack of project knowledge, while GUIRepair_{I2C} compensates for this by using the Image2Code, which improves the overall effectiveness.

3) **Enhanced Capability of the Code2Image Component:** As shown in Table III, GUIRepair_{C2I} presents enhancements of the Code2Image component to the basic agentless approach. Specifically, GUIRepair_{C2I} solves 12 more multimodal task instances than GUIRepair_{base}, achieving a 8.82% improvement. Such results indicate that the Code2Image component is critical for improving repair capabilities. It should be noted that although GUIRepair_{C2I} achieves better results, it is almost three times more expensive to repair than GUIRepair_{base}. The reason for the increase in repair cost is mainly due to the multiple sampling performed by the patch generation in order to fully explore the patch space and the large amount of image information read when receiving visual feedback in patch selection phase. However, such a repair cost is still acceptable and it is still cheaper than many agent systems. Overall, capturing visual effects of repair behaviors to provide fix feedback is an effective validation scheme for visual issues.

4) **Full Capability of the Complete Pipeline:** We have presented the unique contributions of two key components. However, using any single component alone cannot fully unlock the complete performance of GUIRepair. On the one hand, Image2Code-only GUIRepair_{I2C} only helps to understand visual scenarios by mining project-specific knowledge for improving fault localization, ignoring a crucial role (i.e., guided patch validation) of reasoning the code behavior behind the image information. On the other hand, Code2Image-only GUIRepair_{C2I} only works on instances with reproduced code, which means it cannot solve most of the visual problems (only around 17% of instances contain reproduced link). As shown in Table III, the full implementation, GUIRepair_{full}, solves 157 instances, which is better than any other variant. Such results suggest that implementing the full workflow simultaneously is the only way to maximize the fixing benefits. To clarify more clearly how the two components work together, we show an instance *next-4182* that can only be resolved by GUIRepair_{full}. As shown in Figure 10a, the issue report does not provide the complete repro code, which results in GUIRepair_{C2I} unable to implement patch validation. Furthermore, even though GUIRepair_{I2C} is able to generate the repro code, it is not equipped with the Code2Image module, resulting in the inability to fully utilize the repro generation. Therefore, these variants cannot solve this case. Here, we show the reasoning trace of GUIRepair_{full} in Figure 10b. Using Image2Code, the model successfully generates repro code (② **Repro Generation**) for the bug scenario after learning project-specific knowledge (① **Knowledge Mining**) and helps the model locate the correct



bug file *cascader-select.jsx* (❸ File Localization). Then, the Code2Image component uses the previously generated repro code to capture the visual effects presented by the fixing behavior. When the iteration validates to Patch_8 (❹ Patch Selection), the model observes that the bug component renders properly on the web page and considers the patch resolved this issue. In this case, the reproduced code is a key factor in helping to solve the problem by tightly linking the Image2Code and Code2Image components to better understand the bug and validate the patch. Since most visual instances lack reproduction links, the full repair potential can only be unlocked by using both key components together. Overall, GUIRepair requires not only the Image2Code to understand the visual problem and generate the reproduced code, but also the Code2Image to use the reproduced code to capture the visual effect of the fixing behavior, which is an inextricably intertwined process.

In the generalizability experiment, we focus on the performance of GUIRepair on additional projects and base models. We follow the recent study [22] testing the performance of GUIRepair on SWE-bench M dev to investigate whether our

Note: all systems listed here use GPT-4o as the base model.

TABLE V: Results of GUIRepair with different base models.

approach can be generalized to additional projects. We choose GPT-4.1 [72] (flagship GPT model for complex tasks) and o4-mini [73] (faster, more affordable reasoning model) to analyze whether our method can work effectively on other base models. As shown in Table IV, GUIRepair achieves the best results on SWE-bench M dev. In particular, GUIRepair_{full} solves 4 more instances than GUIRepair_{base}, showing that our approach is equally effective in other projects. In addition, we show the performance of GUIRepair on different models in Table V. Specifically, GUIRepair_{full} resolves 21, 13, and 15 more instances than GUIRepair_{base} when using GPT-4o, GPT-4.1, and o4-mini, respectively. Notably, with the reasoning model o4-mini, GUIRepair outperforms the best commercial system, Globant, by 22 instances (175 vs. 153). Such results show that our approach is not only effective across different models, but also significantly benefits from enhanced reasoning capabilities [74], surpassing SOTA commercial solutions. Overall, these findings highlight the generalizability of GUIRepair across diverse projects and model backbones.

Data Leakage. To mitigate the threat of data leakage, we keep the same model as baselines. In the main experiment (Table I), GUIRepair based on GPT-4o still performs better than baselines using the same model. This suggests that even under the same threat of data leakage, our approach still outperforms other schemes. Moreover, our ablation study (Table III) and generalizability study (Table V) show that the improvement does not only stem from the powerful base model, but is further enhanced by the design choices of GUIRepair.

Repo Configuration. We manually install dependencies and deploy runtime environments for each project, which is an inefficient process. Fortunately, SWE-bench M just released

docker images. Besides, we see the potential for autonomous systems to replace humans in deploying and running code repository [75], and we plan to use these automated techniques in the future to replace manual/hard-encoding practices.

VII. RELATED WORK

LLM for APR. Researchers have proposed many LLM-based APR tools to solve software engineering problems. Early researchers fixed real-world software defects by fine-tuning [25], [39], [55], [58], [62]–[64], [76], [77] or prompting [10]–[13], [23], [57], [59]–[61], [65], [78]–[83] LLMs and showed promising results. Recently, researchers [16]–[21], [84]–[87] have begun to focus on github issue resolution [15], [22], and proposed many agent/agentless-based autonomous systems to solve complex repo-level issue scenarios. For example, SWE-agent [17] solves SWE problems by designing Agent Computer Interface to interact with the environment, SpecRover [19] guides the LLM fixing process by specifications inference, and Agentless [21] employs a simplistic pipeline of localization, repair, and validation to solve software development problems, etc. However, most existing SWE systems are not designed for multimodal scenarios, which makes them difficult to solve visual problems [22]. In addition, although some studies also focus on visual-related issues, they are often designed only for specific bug types and are difficult to generalize to generic visual problem domains. For example, DesignRepair [88] only focuses on UI design-related quality issues, Iris [89] is designed for color-related accessibility issues. As shown in Figure 1, multimodal task instances contain various issue scenarios, leading to the need for more generalized solutions to be applied in the vision software domain. GUIRepair builds on simple agentless workflows [18], [30] and enhances the ability to solve multimodal issues by leveraging cross-modal reasoning to understand and capture visual information.

LLM for GUI. GUI-related problems directly affect the user experience. A key area of research is GUI testing, especially researchers have used LLM-driven approaches to implement GUI testing [90], [91]. For example, researchers explored applications by enabling LLMs to understand multimodal scenarios and simulate user actions to test and reproduce GUI-related issues [28], [92], [93]. Especially, recent work has investigated reproducing bugs from bug reports [94], [95] or bug videos [96], [97]. And this key reproduction information has successfully helped to implement soap opera tests generation [98], [99]. Inspired by GUI testing work, GUIRepair implements fault comprehension and patch validation by generating reproduced code and replaying visual scenarios to migrate the experience of GUI testing to APR.

VIII. CONCLUSION

In this work, we propose GUIRepair, a cross-modal reasoning agentless approach that designs Image2Code and Code2Image components for understanding and capturing visual information to enhance the capabilities of autonomous systems for

multimodal SWE problems. Our evaluation on the SWE-bench M demonstrates that GUIRepair can achieve the highest performance compared to other open/closed-source systems.

REFERENCES

- [1] C. Le Goues, M. Pradel, A. Roychoudhury, and S. H. Tan, “Automated Programming and Program Repair (Dagstuhl Seminar 24431),” *Dagstuhl Reports*, vol. 14, no. 10, pp. 39–57, 2025.
- [2] C. Le Goues, M. Pradel, and A. Roychoudhury, “Automated program repair,” *Communications of the ACM (CACM)*, vol. 62, no. 12, pp. 56–65, 2019.
- [3] K. Huang, Z. Xu, S. Yang, H. Sun, X. Li, Z. Yan, and Y. Zhang, “Evolving paradigms in automated program repair: Taxonomy, challenges, and opportunities,” *ACM Computing Surveys (CSUR)*, vol. 57, no. 2, pp. 1–43, 2024.
- [4] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, “A survey of learning-based automated program repair,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 33, no. 2, pp. 1–69, 2023.
- [5] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen, “A systematic literature review on large language models for automated program repair,” *arXiv preprint arXiv:2405.01466*, 2024.
- [6] J. Renzullo, P. Reiter, W. Weimer, and S. Forrest, “Automated program repair: Emerging trends pose and expose problems for benchmarks,” *ACM Computing Surveys (CSUR)*, vol. 57, no. 8, pp. 1–18, 2025.
- [7] H. Eladawy, C. Le Goues, and Y. Brun, “Automated program repair, what is it good for? not absolutely nothing!” in *46th International Conference on Software Engineering (ICSE)*, 2024, pp. 1–13.
- [8] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, Y. Yang, W. Sun, S. Yu, and Z. Chen, “A survey on large language models for software engineering,” *arXiv preprint arXiv:2312.15223*, 2023.
- [9] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, “Large language model-based agents for software engineering: A survey,” *arXiv preprint arXiv:2409.02977*, 2024.
- [10] I. Bouzenia, P. Devanbu, and M. Pradel, “Repairagent: An autonomous, llm-based agent for program repair,” in *47th International Conference on Software Engineering (ICSE)*, 2025, pp. 694–694.
- [11] C. Lee, C. S. Xia, L. Yang, J.-t. Huang, Z. Zhu, L. Zhang, and M. R. Lyu, “A unified debugging approach via llm-based multi-agent synergy,” *arXiv preprint arXiv:2404.17153*, 2024.
- [12] C. S. Xia and L. Zhang, “Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using chatgpt,” in *33rd International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 819–831.
- [13] X. Yin, C. Ni, S. Wang, Z. Li, L. Zeng, and X. Yang, “Thinkrepair: Self-directed automated program repair,” in *33rd International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1274–1286.
- [14] J. Kong, M. Cheng, X. Xie, S. Liu, X. Du, and Q. Guo, “Contrastrepair: Enhancing conversation-based automated program repair via contrastive test case pairs,” *arXiv preprint arXiv:2403.01971*, 2024.
- [15] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?” in *12th International Conference on Learning Representations (ICLR)*, 2024.
- [16] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “Autocoderover: Autonomous program improvement,” in *33rd International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1592–1604.
- [17] J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” *Advances in Neural Information Processing Systems (NIPS)*, vol. 37, pp. 50 528–50 652, 2024.
- [18] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Demystifying llm-based software engineering agents,” *Proc. ACM Softw. Eng. (FSE)*, 2025.
- [19] H. Ruan, Y. Zhang, and A. Roychoudhury, “Specrover: Code intent extraction via llms,” in *47th International Conference on Software Engineering (ICSE)*, 2025, pp. 617–617.
- [20] Y. Ma, R. Cao, Y. Cao, Y. Zhang, J. Chen, Y. Liu, Y. Liu, B. Li, F. Huang, and Y. Li, “Lingma swe-gpt: An open development-process-centric language model for automated software improvement,” in *34th International Symposium on Software Testing and Analysis (ISSTA)*, 2025.
- [21] Y. Ma, Q. Yang, R. Cao, B. Li, F. Huang, and Y. Li, “Alibaba lingmaagent: Improving automated issue resolution via comprehensive repository exploration,” *arXiv preprint arXiv:2406.01422*, 2024.

- [22] J. Yang, C. E. Jimenez, A. L. Zhang, K. Lieret, J. Yang, X. Wu, O. Press, N. Muennighoff, G. Synnaeve, K. R. Narasimhan *et al.*, “Swe-bench multimodal: Do ai systems generalize to visual software domains?” in *13th International Conference on Learning Representations (ICLR)*, 2025.
- [23] H. Ye, A. Z. Yang, C. Hu, Y. Wang, T. Zhang, and C. L. Goues, “Adversarial reasoning for repair based on inferred program intent,” *arXiv preprint arXiv:2505.13008*, 2025.
- [24] R. Just, D. Jalali, and M. D. Ernst, “Defects4j: A database of existing faults to enable controlled testing studies for java programs,” in *2014 International Symposium on Software Testing and Analysis (ISSTA)*, 2014, pp. 437–440.
- [25] N. Jiang, K. Liu, T. Lutellier, and L. Tan, “Impact of code language models on automated program repair,” in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1430–1442.
- [26] R. Cheng, M. Tufano, J. Cito, J. Cambrono, P. Rondon, R. Wei, A. Sun, and S. Chandra, “Agentic bug reproduction for effective automated program repair at google,” *arXiv preprint arXiv:2502.01821*, 2025.
- [27] GUIRepair, “Guirepair,” 2025. [Online]. Available: <https://sites.google.com/view/guirepair>
- [28] S. Feng and C. Chen, “Prompting is all you need: Automated android bug replay with large language models,” in *46th International Conference on Software Engineering (ICSE)*, 2024, pp. 1–13.
- [29] AlibabaFusion/next-895, “label should show in icononly mode,” 2025. [Online]. Available: <https://github.com/alibaba-fusion/next/issues/894>
- [30] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless lite: Rag-based swe-bench software engineering scaffold,” 2025. [Online]. Available: <https://github.com/sorendunn/Agentless-Lite>
- [31] pixelmatch, “The smallest, simplest and fastest javascript pixel-level image comparison library,” 2025. [Online]. Available: <https://github.com/mapbox/pixelmatch>
- [32] Puppeteer, “Javascript api for chrome and firefox,” 2025. [Online]. Available: <https://github.com/puppeteer/puppeteer>
- [33] Chart.js, “Simple yet flexible javascript charting for designers and developers,” 2025. [Online]. Available: <https://github.com/chartjs/Chart.js>
- [34] OpenLayers, “A high-performance, feature-packed library for all your mapping needs,” 2025. [Online]. Available: <https://github.com/openlayers/openlayers>
- [35] chartjs/Chart.js 1509, “borderradius gets ignored for the bottom corners of 0-value bars in bar chart when borderskipped and minbarlength are set,” 2025. [Online]. Available: <https://github.com/chartjs/Chart.js/issues/10005>
- [36] Z. Song, B. Yan, Y. Liu, M. Fang, M. Li, R. Yan, and X. Chen, “Injecting domain-specific knowledge into large language models: a comprehensive survey,” *arXiv preprint arXiv:2502.10708*, 2025.
- [37] S. B. Hossain, R. Taylor, and M. Dwyer, “Doc2oracle: Investigating the impact of javadoc comments on test oracle generation,” *arXiv preprint arXiv:2412.09360*, 2024.
- [38] OpenAI, “Gpt-4o,” 2025. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4o>
- [39] W. Wang, Y. Wang, S. Joty, and S. C. Hoi, “Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair,” in *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*, 2023, pp. 146–158.
- [40] OpenAI, “text-embedding-3-small,” 2025. [Online]. Available: <https://platform.openai.com/docs/models/text-embedding-3-small>
- [41] Z. Lu, Y. Yang, H. Ren, H. Hou, H. Xiao, K. Wang, W. Shi, A. Zhou, M. Zhan, and H. Li, “Webgen-bench: Evaluating llms on generating interactive and functional websites from scratch,” *arXiv preprint arXiv:2505.03733*, 2025.
- [42] Y. Wan, C. Wang, Y. Dong, W. Wang, S. Li, Y. Huo, and M. R. Lyu, “Automatically generating ui code from screenshot: A divide-and-conquer-based approach,” *arXiv preprint arXiv:2406.16386*, 2024.
- [43] AlibabaFusion/next-1509, “The popupcontainer configuration on configprovider does not take effect on dialog,” 2025. [Online]. Available: <https://github.com/alibaba-fusion/next/issues/1508>
- [44] Z. Chen, S. Kommrusch, M. Tufano, L.-N. Pouchet, D. Poshvanyk, and M. Monperrus, “Sequencer: Sequence-to-sequence learning for end-to-end program repair,” *IEEE Transactions on Software Engineering (TSE)*, vol. 47, no. 9, pp. 1943–1959, 2019.
- [45] H. Ye, M. Martinez, and M. Monperrus, “Neural program repair with execution-based backpropagation,” in *44th International Conference on Software Engineering (ICSE)*, 2022, pp. 1506–1518.
- [46] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, “Selfapr: Self-supervised program repair with test execution diagnostics,” in *37th International Conference on Automated Software Engineering (ASE)*, 2022, pp. 1–13.
- [47] H. Ye and M. Monperrus, “Iter: Iterative neural repair for multi-location patches,” in *46th International Conference on Software Engineering (ICSE)*, 2024, pp. 1–13.
- [48] Y. Li, S. Wang, and T. N. Nguyen, “Dlfix: Context-based code transformation learning for automated program repair,” in *42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 602–614.
- [49] —, “Dear: A novel deep learning-based approach for automated program repair,” in *44th International Conference on Software Engineering (ICSE)*, 2022, pp. 511–523.
- [50] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, “Coconut: combining context-aware neural translation models using ensemble for program repair,” in *29th International Symposium on Software Testing and Analysis (ISSTA)*, 2020, pp. 101–114.
- [51] N. Jiang, T. Lutellier, and L. Tan, “Cure: Code-aware neural machine translation for automatic program repair,” in *43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 1161–1173.
- [52] N. Jiang, T. Lutellier, Y. Lou, L. Tan, D. Goldwasser, and X. Zhang, “Knod: Domain knowledge distilled tree decoder for automated program repair,” in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1251–1263.
- [53] Q. Zhu, Z. Sun, Y.-a. Xiao, W. Zhang, K. Yuan, Y. Xiong, and L. Zhang, “A syntax-guided edit decoder for neural program repair,” in *29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*, 2021, pp. 341–353.
- [54] Q. Zhu, Z. Sun, W. Zhang, Y. Xiong, and L. Zhang, “Tare: Type-aware neural program repair,” in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1443–1455.
- [55] K. Huang, J. Zhang, X. Meng, and Y. Liu, “Template-guided program repair in the era of large language models,” in *47th International Conference on Software Engineering (ICSE)*, 2025, pp. 367–379.
- [56] Y. Wei, C. S. Xia, and L. Zhang, “Copiloting the copilots: Fusing large language models with completion engines for automated program repair,” in *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*, 2023, pp. 172–184.
- [57] C. S. Xia and L. Zhang, “Less training, more repairing please: revisiting automated program repair via zero-shot learning,” in *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*, 2022, pp. 959–971.
- [58] C. S. Xia, Y. Ding, and L. Zhang, “The plastic surgery hypothesis in the era of large language models,” in *38th International Conference on Automated Software Engineering (ASE)*, 2023, pp. 522–534.
- [59] J. Xiang, X. Xu, F. Kong, M. Wu, Z. Zhang, H. Zhang, and Y. Zhang, “How far can we go with practical function-level program repair?” *arXiv preprint arXiv:2404.12833*, 2024.
- [60] J. Xu, Y. Fu, S. H. Tan, and P. He, “Aligning the objective of llm-based program repair,” *arXiv preprint arXiv:2404.08877*, 2024.
- [61] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, “Automated repair of programs from large language models,” in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1469–1481.
- [62] K. Huang, X. Meng, J. Zhang, Y. Liu, W. Wang, S. Li, and Y. Zhang, “An empirical study on fine-tuning large language models of code for automated program repair,” in *38th International Conference on Automated Software Engineering (ASE)*, 2023, pp. 1162–1174.
- [63] K. Huang, J. Zhang, X. Bao, X. Wang, and Y. Liu, “Comprehensive fine-tuning large language models of code for automated program repair,” *IEEE Transactions on Software Engineering (TSE)*, pp. 1–25, 2025.
- [64] Y. Wu, N. Jiang, H. V. Pham, T. Lutellier, J. Davis, L. Tan, P. Babkin, and S. Shah, “How effective are neural networks for fixing security vulnerabilities,” in *32nd International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 1282–1294.
- [65] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1482–1494.
- [66] Prism/prism-1602, “Yaml strings fail with trailing comments,” 2025. [Online]. Available: <https://github.com/PrismJS/prism/issues/1601>
- [67] SWE-bench, “Swe-bench multimodal leaderboard,” 2025. [Online]. Available: <https://www.swebench.com/index.html#multimodal>
- [68] P. Aggarwal and S. Welleck, “Programming with pixels: Computer-use meets software engineering,” *arXiv preprint arXiv:2502.18525*, 2025.

- [69] Playwright, “Playwright enables reliable end-to-end testing for modern web apps,” 2025. [Online]. Available: <https://playwright.dev/>
- [70] Globant, “Globant code fixer agent,” 2025. [Online]. Available: <https://ai.globant.com/>
- [71] Zencoder, “The ai coding agent,” 2025. [Online]. Available: <https://zencoder.ai/>
- [72] OpenAI, “Gpt-4.1,” 2025. [Online]. Available: <https://platform.openai.com/docs/models/gpt-4.1>
- [73] —, “o4-mini,” 2025. [Online]. Available: <https://platform.openai.com/docs/models/o4-mini>
- [74] Y. Wei, O. Duchenne, J. Copet, Q. Carbonneaux, L. Zhang, D. Fried, G. Synnaeve, R. Singh, and S. I. Wang, “Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution,” *arXiv preprint arXiv:2502.18449*, 2025.
- [75] I. Bouzenia and M. Pradel, “You name it, i run it: An llm agent to execute tests of arbitrary projects,” *arXiv preprint arXiv:2412.10133*, 2024.
- [76] B. Yang, H. Tian, J. Ren, H. Zhang, J. Klein, T. Bissyande, C. Le Goues, and S. Jin, “Morepair: Teaching llms to repair code via multi-objective fine-tuning,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2025.
- [77] A. Silva, S. Fang, and M. Monperrus, “Repairllama: Efficient representations and fine-tuned adapters for program repair,” *arXiv preprint arXiv:2312.15698*, 2023.
- [78] Q. Zhang, C. Fang, T. Zhang, B. Yu, W. Sun, and Z. Chen, “Gamma: Revisiting template-based automated program repair via mask prediction,” in *38th International Conference on Automated Software Engineering (ASE)*, 2023, pp. 535–547.
- [79] H. Zheng, I. Shumailov, T. Fan, A. Hall, and M. Payer, “Fixing 7,400 bugs for 1\$: Cheap crash-site program repair,” *arXiv preprint arXiv:2505.13103*, 2025.
- [80] J. Zhao, D. Yang, L. Zhang, X. Lian, Z. Yang, and F. Liu, “Enhancing automated program repair with solution design,” in *39th International Conference on Automated Software Engineering (ASE)*, 2024, pp. 1706–1718.
- [81] B. Yang, H. Tian, W. Pian, H. Yu, H. Wang, J. Klein, T. F. Bissyandé, and S. Jin, “Cref: An llm-based conversational software repair framework for programming tutors,” in *33rd International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 882–894.
- [82] Y. Xiao, W. Wang, D. Liu, J. Zhou, S. Cheng, and Y. Xiong, “Predicatefix: Repairing static analysis alerts with bridging predicates,” *arXiv preprint arXiv:2503.12205*, 2025.
- [83] Y. Peng, S. Gao, C. Gao, Y. Huo, and M. Lyu, “Domain knowledge matters: Improving prompts with fix templates for repairing python type errors,” in *46th International Conference on Software Engineering (ICSE)*, 2024, pp. 1–13.
- [84] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh *et al.*, “Openhands: An open platform for ai software developers as generalist agents,” in *The Thirteenth International Conference on Learning Representations*, 2024.
- [85] A. Antoniadou, A. Örwall, K. Zhang, Y. Xie, A. Goyal, and W. Wang, “Swe-search: Enhancing software agents with monte carlo tree search and iterative refinement,” *arXiv preprint arXiv:2410.20285*, 2024.
- [86] Y. Liu, P. Gao, X. Wang, J. Liu, Y. Shi, Z. Zhang, and C. Peng, “Marscode agent: Ai-native automated bug fixing,” *arXiv preprint arXiv:2409.00899*, 2024.
- [87] X. Meng, Z. Ma, P. Gao, and C. Peng, “An empirical study on llm-based agents for automated bug fixing,” *arXiv preprint arXiv:2411.10213*, 2024.
- [88] M. Yuan, J. Chen, Z. Xing, A. Quigley, Y. Luo, T. Luo, G. Mohammadi, Q. Lu, and L. Zhu, “Designrepair: Dual-stream design guideline-aware frontend repair with large language models,” *arXiv preprint arXiv:2411.01606*, 2024.
- [89] Y. Zhang, S. Chen, L. Fan, C. Chen, and X. Li, “Automated and context-aware repair of color-related accessibility issues for android apps,” in *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*, 2023, pp. 1255–1267.
- [90] F. Tang, H. Xu, H. Zhang, S. Chen, X. Wu, Y. Shen, W. Zhang, G. Hou, Z. Tan, Y. Yan *et al.*, “A survey on (m) llm-based gui agents,” *arXiv preprint arXiv:2504.13865*, 2025.
- [91] S. Yu, C. Fang, Z. Tuo, Q. Zhang, C. Chen, Z. Chen, and Z. Su, “Vision-based mobile app gui testing: A survey,” *arXiv preprint arXiv:2310.13518*, 2023.
- [92] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, “Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions,” in *46th International Conference on Software Engineering (ICSE)*, 2024, pp. 1–13.
- [93] Y. Su, Z. Xing, C. Wang, C. Chen, X. Xu, Q. Lu, and L. Zhu, “Automated soap opera testing directed by llms and scenario knowledge: Feasibility, challenges, and road ahead,” *arXiv preprint arXiv:2412.08581*, 2024.
- [94] M. Fazzini, M. Prammer, M. d’Amorim, and A. Orso, “Automatically translating bug reports into test cases for mobile apps,” in *27th International Symposium on Software Testing and Analysis (ISSTA)*, 2018, pp. 141–152.
- [95] Y. Zhao, T. Yu, T. Su, Y. Liu, W. Zheng, J. Zhang, and W. G. Halfond, “Recdroid: automatically reproducing android application crashes from bug reports,” in *41st International Conference on Software Engineering (ICSE)*, 2019, pp. 128–139.
- [96] C. Bernal-Cárdenas, N. Cooper, K. Moran, O. Chaparro, A. Marcus, and D. Poshyanyk, “Translating video recordings of mobile app usages into replayable scenarios,” in *42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 309–321.
- [97] M. Havranek, C. Bernal-Cárdenas, N. Cooper, O. Chaparro, D. Poshyanyk, and K. Moran, “V2s: A tool for translating video recordings of mobile app usages into replayable scenarios,” in *43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2021, pp. 65–68.
- [98] Y. Su, Z. Han, Z. Xing, X. Xia, X. Xu, L. Zhu, and Q. Lu, “Constructing a system knowledge graph of user tasks and failures from bug reports to support soap opera testing,” in *37th International Conference on Automated Software Engineering (ASE)*, 2022, pp. 1–13.
- [99] Y. Su, Z. Han, Z. Xing, X. Xu, L. Zhu, and Q. Lu, “Soapoperatg: A tool for system knowledge graph based soap opera test generation,” in *45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2023, pp. 51–54.